

Assignment #9: dfs, bfs, & dp

2024 fall, Compiled by 吴诚舟---物理学院

说明:

- 1) 请把每个题目解题思路（可选），源码Python, 或者C++（已经在Codeforces/Openjudge上AC），截图（包含Accepted），填写到下面作业模版中（推荐使用 typora <https://typoraio.cn>，或者用 word）。AC 或者没有AC，都请标上每个题目大致花费时间。
- 2) 提交时候先提交pdf文件，再把md或者doc文件上传到右侧“作业评论”。Canvas需要有同学清晰头像、提交文件有pdf、“作业评论”区有上传的md或者doc附件。
- 3) 如果不能在截止前提交作业，请写明原因。

1. 题目

18160: 最大连通域面积

dfs similar, <http://cs101.openjudge.cn/practice/18160>

代码:

```
dxs = [-1, 0, 1]; dys = [-1, 0, 1]
def dfs(field, x, y, area = 1):
    for dx in dxs:
        for dy in dys:
            nx, ny = x+dx, y+dy
            if field[nx][ny] == 'w':
                field[nx][ny] = '.'
                area += 1
            area = dfs(field, nx, ny, area)
    return area

t = int(input())
answer = []
for _ in range(t):
    n, m = map(int, input().split())
    largest_area = 0
    field = [['.']*m]
    field = field+[['.']+list(input())+['.'] for _ in range(n)]
    field = field+[['.']*m]
    for i in range(1, n+1):
        for j in range(1, m+1):
            if field[i][j] == 'w':
                field[i][j] = '.'
                this_area = dfs(field, i, j)
                if this_area > largest_area:
                    largest_area = this_area
    answer.append(largest_area)
for ans in answer:
```

```
print(ans)
```

代码运行截图 (至少包含有"Accepted")

#47281043提交状态

查看提交统计提问

状态: Accepted

源代码

```
dxs = [-1, 0, 1]; dys = [-1, 0, 1]
def dfs(field, x, y, area = 1):
    for dx in dxs:
        for dy in dys:
            nx, ny = x+dx, y+dy
            if field[nx][ny] == 'W':
                field[nx][ny] = '.'
                area += 1
            area = dfs(field, nx, ny, area)
    return area

t = int(input())
answer = []
for _ in range(t):
    n, m = map(int, input().split())
    largest_area = 0
    field = [['.']*m+2]
    field = field+[['.']+list(input())+['.']] for _ in range(n)
    field = field+[['.']*m+2]
    for i in range(1, n+1):
        for j in range(1, m+1):
            if field[i][j] == 'W':
                field[i][j] = '.'
                this_area = dfs(field, i, j)
                if this_area > largest_area:
                    largest_area = this_area
    answer.append(largest_area)
for ans in answer:
    print(ans)
```

基本信息

#: 47281043

题目: 18160

提交人: 24n2400011447

内存: 3696kB

时间: 77ms

语言: Python3

提交时间: 2024-11-20 13:34:16

19930: 寻宝

bfs, <http://cs101.openjudge.cn/practice/19930>

调试了很久才考虑到field[1][1]==1的坑点

代码:

```
from collections import deque
direction = [(1, 0), (0, 1), (-1, 0), (0, -1)]
step = 0

def bfs(field):
    global step

    if field[1][1] == 1:
        step = 0
        return
    queue = [(1, 1)]; field[1][1] = 2
    while queue:
        buffer = deque(queue)
        queue = []
        step += 1
        while buffer:
            x, y = buffer.popleft()
```

```

        for i in range(4):
            nx, ny = x+direction[i][0], y+direction[i][1]
            if field[nx][ny] == 1:
                return
            elif field[nx][ny] == 0:
                field[nx][ny] = 2
                queue.append((nx, ny))

    step = 'NO'

m, n = map(int, input().split())

field = [[2]*(n+2)]
field = field + [[2]+list(map(int, input().split()))+2 for _ in range(m)]
field = field + [[2]*(n+2)]

bfs(field)
print(step)

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

源代码

```

from collections import deque
direction = [(1, 0), (0, 1), (-1, 0), (0, -1)]
step = 0

def bfs(field):
    global step

    if field[1][1] == 1:
        step = 0
        return
    queue = [(1, 1)]; field[1][1] = 2
    while queue:
        buffer = deque(queue)
        queue = []
        step += 1
        while buffer:
            x, y = buffer.popleft()
            for i in range(4):
                nx, ny = x+direction[i][0], y+direction[i][1]
                if field[nx][ny] == 1:
                    return
                elif field[nx][ny] == 0:
                    field[nx][ny] = 2
                    queue.append((nx, ny))

    step = 'NO'

m, n = map(int, input().split())

field = [[2]*(n+2)]
field = field + [[2]+list(map(int, input().split()))+2 for _ in range
field = field + [[2]*(n+2)]

bfs(field)
print(step)

```

基本信息

#: 47282990

题目: 19930

提交人: 24n2400011447

内存: 3712kB

时间: 33ms

语言: Python3

提交时间: 2024-11-20 14:27:26

04123: 马走日

dfs, <http://cs101.openjudge.cn/practice/04123>

代码:

```

directions = [(2, 1), (1, 2), (-2, 1), (-1, 2), (2, -1), (1, -2), (-2, -1), (-1,
-2)]

```

```

def dfs(field, x, y, res_pos, routes = 0):
    if res_pos == 0:
        routes += 1
        return routes
    for i in range(8):
        nx, ny = x + directions[i][0], y + directions[i][1]
        if field[nx+1][ny+1] == 0:
            field[nx+1][ny+1] = 1
            routes = dfs(field, nx, ny, res_pos-1, routes)
            field[nx+1][ny+1] = 0
    return routes

t = int(input())
answer = []
for _ in range(t):
    n, m, x, y = map(int, input().split())

    field = [[1]*(m+4) for _ in range(2)]
    field = field + [[1, 1]+[0]*m+[1, 1] for _ in range(n)]
    field = field + [[1]*(m+4) for _ in range(2)]

    field[x+2][y+2] = 1
    routes = dfs(field, x+1, y+1, m*n-1)
    answer.append(routes)

for ans in answer:
    print(ans)

```

代码运行截图 (至少包含有"Accepted")

状态: Accepted

源代码

```

directions = [(2, 1), (1, 2), (-2, 1), (-1, 2), (2, -1), (1, -2), (-2, -1), (-1, -2)]
def dfs(field, x, y, res_pos, routes = 0):
    if res_pos == 0:
        routes += 1
        return routes
    for i in range(8):
        nx, ny = x + directions[i][0], y + directions[i][1]
        if field[nx+1][ny+1] == 0:
            field[nx+1][ny+1] = 1
            routes = dfs(field, nx, ny, res_pos-1, routes)
            field[nx+1][ny+1] = 0
    return routes

t = int(input())
answer = []
for _ in range(t):
    n, m, x, y = map(int, input().split())

    field = [[1]*(m+4) for _ in range(2)]
    field = field + [[1, 1]+[0]*m+[1, 1] for _ in range(n)]
    field = field + [[1]*(m+4) for _ in range(2)]

    field[x+2][y+2] = 1
    routes = dfs(field, x+1, y+1, m*n-1)
    answer.append(routes)

for ans in answer:
    print(ans)

```

基本信息

#: 47284241
 题目: 04123
 提交人: 24n2400011447
 内存: 3652kB
 时间: 3486ms
 语言: Python3
 提交时间: 2024-11-20 15:08:55

sy316: 矩阵最大权值路径

dfs, <https://sunnywhy.com/sfbj/8/1/316>

代码:

```
direction = [(1, 0), (0, 1), (-1, 0), (0, -1)]

def dfs(field, x, y):
    global val, mval
    global final_route
    for i in range(4):
        nx, ny = x+direction[i][0], y+direction[i][1]
        if nx == n and ny == m:
            route.append((n, m))
            if val + value[n][m] > mval:
                mval = val + value[n][m]
                final_route = route[:]
            route.pop()
            continue
        if field[nx][ny] == 0:
            val += value[nx][ny]
            route.append((nx, ny))
            field[nx][ny] = 1
            dfs(field, nx, ny)
            field[nx][ny] = 0
            route.pop()
            val -= value[nx][ny]
    return

n, m = map(int, input().split())

value = [[1]*(m+2)]
value = value+[[1]+list(map(int, input().split()))+[1] for _ in range(n)]
value = value+[[1]*(m+2)]
field = [[1]*(m+2)]+[[1]+[0]*m+[1] for _ in range(n)]+[[1]*(m+2)]

field[1][1] = 1
final_route = []
route = [(1, 1)]
val = value[1][1]; mval = float('-inf')

dfs(field, 1, 1)
for point in final_route:
    print(*point)
```

代码运行截图 (至少包含有"Accepted")

输入描述

第一行两个整数 n, m ($2 \leq n \leq 5, 2 \leq m \leq 5$), 分别表示矩阵的行数和列数;
接下来 n 行, 每行 m 个整数 ($-100 \leq \text{整数} \leq 100$), 表示矩阵每个位置的权值。

输出描述

从左上角的坐标开始, 输出若干行 (每行两个整数, 表示一个坐标), 直到右下角的坐标。
数据保证权值之和最大的路径存在且唯一。

样例1

输入

2 2
1 2
3 4

输出

1 1
2 1
2 2

解释

显然当路径是 $(1,1) \Rightarrow (2,1) \Rightarrow (2,2)$ 时, 权值之和最大, 即 $1 + 3 + 4 = 8$ 。

```
19 def dfs(field, nx, ny):
20     field[nx][ny] = 0
21     route.pop()
22     val -= value[nx][ny]
23     return
24
25
26 n, m = map(int, input().split())
27
28 value = [[1]*(m+2)]
29 value = value+[[1]+list(map(int, input().split()))+[1] for _ in range(n)
30 value = value+[[1]*(m+2)]
31 field = [[1]*(m+2)]+[[1]+[0]*m+[1] for _ in range(n)]+[[1]*(m+2)]
32
33 field[1][1] = 1
34 final_route = []
35 route = [(1, 1)]
36 val = value[1][1]; mval = float('-inf')
37
38 dfs(field, 1, 1)
39 for point in final_route:
40     print(*point)
```

测试输入 提交结果 历史提交

完美通过 100% 数据通过测试 运行时长: 0 ms 查看题解

LeetCode62.不同路径

dp, <https://leetcode.cn/problems/unique-paths/>

代码:

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [[1]*n]+[[1]+[0]*(n-1) for _ in range(m-1)]

        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i][j-1]+dp[i-1][j]
        return dp[-1][-1]
```

代码运行截图 (至少包含有"Accepted")

题目描述 通过 题解 提交记录

全部提交记录

通过 Distracted GangulyuM9 提交于 2024.11.20 16:04 官方题解 写题解

面向在校学生的专属优惠 完成认证至 7 新升级 Plus 会员, 日均 1 元

执行用时分布 0 ms 击败 100.00% 复杂度分析

消耗内存分布 16.39 MB 击败 72.54%

代码 Python3

class Solution:
 def uniquePaths(self, m: int, n: int) -> int:
 dp = [[1]*n]+[[1]+[0]*(n-1) for _ in range(m-1)]

 for i in range(1, m):
 for j in range(1, n):
 dp[i][j] = dp[i][j-1]+dp[i-1][j]
 return dp[-1][-1]

Python3 智能模式

1 class Solution:
2 def uniquePaths(self, m: int, n: int) -> int:
3 dp = [[1]*n]+[[1]+[0]*(n-1) for _ in range(m-1)]
4
5 for i in range(1, m):
6 for j in range(1, n):
7 dp[i][j] = dp[i][j-1]+dp[i-1][j]
8 return dp[-1][-1]

行 8, 列 26 | 已存储 运行 提交

测试用例 测试结果

Case 1 Case 2 +

m = 3
n = 7

Source

sy358: 受到祝福的平方

dfs, dp, <https://sunnywhy.com/sfbj/8/3/539>

代码：

```
square_number = set()
for i in range(1, 35000):
    square_number.add(i*i)

def is_splendid(a):
    if int(a) in square_number:
        return True
    for i in range(1, len(a)):
        part1 = a[:i]
        part2 = a[i:]
        if int(part1) in square_number:
            if is_splendid(part2):
                return True
    return False

print('Yes') if is_splendid(input()) else print('No')
```

代码运行截图 (至少包含有"Accepted")

题目 题解

在小元的世界里，任何人出生后会世界分配一个随机ID，如果ID在被切割后，即ID满足按照从左至右顺序分割，且分割出来的数字都是某一个正整数的平方，分割时可以包括前导0，那么他就被这个世界祝福，最后获得快乐的质量和数量都比不满足这样的ID的人多。

令ID为A，且A是一个正整数，取值范围为 $1 \leq A \leq 10^9$ ，问A是否是一个被受到祝福的ID。

比如A = 8194时，它是一个被受到祝福的ID，因为他可以被分割为 $\{81, 9, 4\} = \{9^2, 3^2, 2^2\}$ ；

比如A = 1001时，它是一个被受到祝福的ID，因为他可以被分割为 $\{1, 001\} = \{1^2, 1^2\}$ ，或者 $\{100, 1\} = \{10^2, 1^2\}$ 。注意 $\{1, 00, 1\} = \{1^2, 0^2, 1^2\}$ 不是一个合法切割，因为分割出来的数字必须为正整数的平方；

比如A = 36时，36已经是一个平方数了，所以它同样满足条件；

比如A = 54，它不是一个被受到祝福的ID，因为他无法被切割为满足条件的集合。

输入描述

一个正整数A，无前导0。

其中 $1 \leq A \leq 10^9$

输出描述

如果是一个满足题目的数字则输出 Yes，否则 No。

代码书写

```
1 square_number = set()
2 for i in range(1, 35000):
3     square_number.add(i*i)
4
5 def is_splendid(a):
6     if int(a) in square_number:
7         return True
8     for i in range(1, len(a)):
9         part1 = a[:i]
10        part2 = a[i:]
11        if int(part1) in square_number:
12            if is_splendid(part2):
13                return True
14    return False
15
16 print('Yes') if is_splendid(input()) else print('No')
```

测试输入 提交结果 历史提交

完美通过

100% 数据通过测试

运行时长: 0 ms

查看题解

2. 学习总结和收获

如果作业题目简单，有否额外练习题目，比如：OJ“计概2024fall每日选做”、CF、LeetCode、洛谷等网站题目。

每日选做在跟进。会写dfs和bfs，但调试仍然要花很长时间（顺利的话20min）。

